

heartBeatEVT

REFERENCE

API consumer

interpret the reader's periodic heartbeat event

Overview

`heartBeatEVT` is a **device-emitted** event — the reader publishes it on its own, with **no command triggering it**, to signal that it is alive and to report basic health: uptime, an event sequence number, and optional **inventory** and **battery** sub-payloads. It is read-only telemetry and carries no credentials.

You **enable** a heartbeat rather than requesting it. The reader publishes `heartBeatEVT` at the interval you set in `heartbeatConfiguration` once the heartbeat is enabled with `config_events` (see [Event Semantics](#)).

The reader publishes `heartBeatEVT` at the configured interval (at most 65,535 seconds) while the heartbeat is enabled with `config_events`; a reboot applies the setting. Monitoring can rely on it as its primary liveness signal: when `heartBeatEVT` stops arriving, the reader, its session, or the path to your application needs attention. `heartBeatEVT` is published at QoS 0, so the MQTT layer does not resend one lost in transit. Choose how many missed intervals your application treats as an outage.

Details

Property	Value
Event Type	Heartbeat (periodic liveness / health)
Trigger	Timed interval (every <code>heartbeatConfiguration.interval</code> s while enabled)
Enabled By	<code>config_events</code> with <code>eventConfiguration.heartbeat: true</code> (plus <code>heartbeatConfiguration</code>), applied on <code>reboot</code>
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
Credentials	None (read-only telemetry, nothing to mask)
Interaction Pattern	Publish / Subscribe (device-emitted, unsolicited; no request/response, no <code>response.code</code>)
Related	<code>dataEVT</code> , <code>alerts</code> , <code>mqttConnEVT</code>

When to Use

Subscribe to `heartBeatEVT` when you want **continuous, pushed telemetry of reader liveness and health** — uptime, an event sequence number, and optional inventory and battery status — rather than polling for it on demand. It is the asynchronous twin of `get_status` (`heartBeatEVT` ↔ `get_status`): the heartbeat *pushes* health at a fixed interval, while `get_status` *pulls* a correlated snapshot when you ask.

- **Receive:** periodic `heartBeatEVT` payloads on the management event topic once the heartbeat is enabled and the reader has rebooted.
- **Use it for:** liveness monitoring (the *absence* of an expected heartbeat is itself a signal), tracking RFID inventory progression, and routine battery monitoring — without issuing a command per check.
- **Prefer a different call when:** you need an immediate, on-demand reading — call `get_status` instead. It returns a synchronous device-health and battery snapshot in a correlated response (note it does not include the heartbeat's inventory scan counts or uptime).

Topic Mapping

`heartBeatEVT` is device-emitted and flows on the **management event topic** — it has no `cmd` / `resp` topics of its own because no command produces it.

Message	Direction	Topic	Note
Event	Reader → App	<code><tenantId>/MGMT/clients/event/<serial></code>	Published on the management endpoint's event topic.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

The event arrives on the same management-event plane you enabled it on with `config_events`.

See [Topics and Channels](#).

Key Fields

- `eventName` (event) — names the event; the fixed value `heartbeat`. Required.
- `timestamp` (event) — ISO-8601 time the heartbeat was generated. Required.
- `eventNumber` (event) — a unique identifier or sequence number for the event. The schema does not guarantee it is monotonic or contiguous, so a gap between consecutive heartbeats is not necessarily a missed heartbeat. Required.
- `data.inventoryStatus` (event) — RFID scan state (`rfidStatus`), plus `tagCount` and `scanCount`. Present when `heartbeatConfiguration.inventoryStatus` is `true`.
- `data.batteryAlert` (event) — battery `status`, `stateOfHealth`, and `chargePercentage`. Present when `heartbeatConfiguration.batteryStatus` is `true`.
- `heartbeat` (request) — the gate flag in `config_events`; the heartbeat publishes only once `heartbeat: true` and `heartbeatConfiguration.interval` is set (at most 65,535 seconds) (see [Event Semantics](#)).

Event Payload

The top-level shape is `eventName`, `timestamp`, `eventNumber` (all **required**), an optional `upTime`, and an optional `data` object carrying the inventory and/or battery sub-payloads. Use the **Schema** tab for the field reference and the **Example** tab for a full heartbeat.

Event payload

Example: inventory + battery

```
1  {
2    "eventName": "heartbeat",
3    "timestamp": "2019-08-24T14:15:22Z",
4    "eventNumber": 120,
5    "data": {
6      "inventoryStatus": {
7        "rfidStatus": "INPROGRESS",
8        "tagCount": 45,
9        "scanCount": 128
10     },
11     "batteryAlert": {
12       "status": "HIGH",
13       "stateOfHealth": "GOOD",
14       "chargePercentage": 85
15     }
16   }
17 }
```

Event payload

★ required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	eventName ★	string	—	heartbeat	Names the event — the fixed value heartbeat for this event.
2	timestamp ★	string	—	2019-08-24T14:15:22Z	ISO-8601 time the event was generated.
3	eventNumber ★	integer	—	120	A unique identifier or sequence number for the event. The schema does not guarantee it is monotonic or contiguous per heartbeat.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
4	upTime	string	—	5 days 4hr 3min	Device uptime when the event was generated. Free-form human-readable string (not a duration type).
5	data	object	—	—	Optional health payload. The schema declares it as anyOf two flat shapes — the inventory fields or the battery fields below — while the contract's example nests them as data.inventoryStatus and data.batteryAlert (see Field Support). Which appear is gated by heartbeatConfiguration (see Event Semantics).
6	rfidStatus	enum	INPROGRESS STOPPED	—	The current status of the RFID scanning process.
7	tagCount	integer	—	—	The total number of RFID tags scanned in the current operation.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
8	s c a n c o u n t	integer	—	—	The total number of scan attempts made during the operation.
9	s t a t u s	enum	LOW CRITICAL CHARGING FULL HIGH	LOW	The current status of the battery, which indicates its operational state.
10	s t a t e o f H e a l t h	enum	GOOD CRITICAL MODERATE	—	The overall health of the battery, which reflects its long-term capacity and performance.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
11	chargePercentage	integer	max 100	—	The current charge level of the battery, represented as a percentage.

In the examples, `data` nests its sub-payloads, and a heartbeat may carry **one, both, or neither** of `inventoryStatus` and `batteryAlert` (the schema declares a flatter shape — see [Field Support](#)). The `heartbeatConfiguration` inclusion flags you set when enabling the heartbeat control which ones appear:

- `inventoryStatus` — RFID scan state (`rfidStatus`), plus `tagCount` and `scanCount` . Present when `heartbeatConfiguration.inventoryStatus` is `true` .
- `batteryAlert` — battery `status` , `stateOfHealth` , and `chargePercentage` . Present when `heartbeatConfiguration.batteryStatus` is `true` .

Event Semantics

Emission

Enable the heartbeat with `config_events` under `eventConfiguration` : set `heartbeat` to `true` and `heartbeatConfiguration.interval` to a value of at most 65,535 seconds. The contract documents no default for either. After the reader has rebooted, it publishes `heartBeatEVT` every `heartbeatConfiguration.interval` seconds on the management event topic.

Field	Type	Meaning
<code>heartbeat</code>	boolean	Set <code>true</code> to enable the heartbeat stream.
<code>heartbeatConfiguration.interval</code>	integer (seconds)	Seconds between heartbeats. Lower = more telemetry, higher battery cost.

Enable the heartbeat via config_events

```

1  {
2    "command" : "config_events" ,
3    "requestId" : "abc123" ,
4    "eventConfiguration" : {
5      "terminalConnection" : true ,
6      "firmwareUpdate" : true ,
7      "network" : true ,
8      "heartbeat" : true ,
9      "power" : true ,
10     "battery" : true ,
11     "temperature" : true ,
12     "fileDownload" : true ,
13     "heartbeatConfiguration" : {
14       "interval" : 100 ,
15       "inventoryStatus" : true ,
16       "batteryStatus" : true
17     }
18   }
19 }
```

`config_events` stores the heartbeat configuration immediately (confirm it with `get_endpoint_config`), but the reader begins **emitting** only after a `reboot`. Also note `heartbeatConfiguration` appears in a `get_endpoint_config` read only after `config_events` sets it — it is absent from a default read. See `config_events` for the full enable workflow.

- **If no heartbeat arrives.** Read the endpoint configuration with `get_endpoint_config` and confirm `heartbeat: true` and an `interval` of at most 65,535. Confirm that the reader has rebooted since you sent `config_events`, and that you subscribed to the management event topic; a heartbeat published before your subscription is active is not delivered later.

Shaping

The `heartbeatConfiguration` inclusion flags shape each heartbeat's optional `data` object — they select which sub-payloads a heartbeat carries, independently of the enabling above.

Field	Type	Meaning
<code>heartbeatConfiguration.inventoryStatus</code>	boolean	Include the <code>data.inventoryStatus</code> block in each heartbeat.
<code>heartbeatConfiguration.batteryStatus</code>	boolean	Include the <code>data.batteryAlert</code> block in each heartbeat.
<code>heartbeatConfiguration.userApps</code>	boolean	Declared for a <code>data.userApps</code> block. Not supported: firmware does not implement user-application status, so no heartbeat carries it.

Because every member of `data` is optional, a heartbeat carries one, both, or neither of `inventoryStatus` and `batteryAlert`, depending on these flags. Set them alongside the enable flags via [`config\_events`](#).

Field Support

- **The schema and the example disagree on the shape of `data`.** The schema declares `data` as `anyOf` two flat objects — `rfidStatus` / `tagCount` / `scanCount`, or `status` / `stateOfHealth` / `chargePercentage` — so those fields would sit directly under `data`. The contract's own example, like every example on this page, nests them as `data.inventoryStatus` and `data.batteryAlert`; it validates only because neither branch forbids extra keys. Parse the nested form shown here, and tolerate the flat form.
- **`stateOfHealth` is not a charge band.** `batteryAlert.stateOfHealth` is enumerated `GOOD` / `CRITICAL` / `MODERATE`. `FULL` is a value of `status`, not of `stateOfHealth`, so a payload carrying `stateOfHealth: "FULL"` is outside the enum. Trust the enum.
- **`batteryAlert.stateOfHealth` is the same health grade as `get_status`.** The value set here is `GOOD` / `CRITICAL` / `MODERATE`, and `get_status` reports `deviceStatus.batteryStatus.stateOfHealth` from that same enum. One grade, so nothing needs translating when you share battery logic between the heartbeat and the command. The battery's charge state is the separate `status` field. See Interpret `stateOfHealth`.

- **Required fields and** `upTime` , `eventName` , `timestamp` , and `eventNumber` are required; `upTime` and `data` are optional. `upTime` is a **free-form string** (e.g. `"5 days 4hr 3min"`), not a machine-parseable duration — do not parse it as a number.

Examples

A heartbeat carrying **both** sub-objects appears in the **Example** tab of the payload above. The variants below show the minimal required-only shape and an inventory-only heartbeat.

The minimum a heartbeat must carry (the three required fields; `data` and `upTime` omitted):

heartBeatEVT — minimal (required fields only)

```

1      {
2          "eventName" : "heartbeat" ,
3          "timestamp" : "2019-08-24T14:15:22Z" ,
4          "eventNumber" : 121
5      }
```

An **inventory-only** heartbeat (`heartbeatConfiguration.inventoryStatus: true` , `batteryStatus: false`):

heartBeatEVT — inventory status only

```

1      {
2          "eventName" : "heartbeat" ,
3          "timestamp" : "2019-08-24T14:15:37Z" ,
4          "eventNumber" : 122 ,
5          "data" : {
6              "inventoryStatus" : {
7                  "rfidStatus" : "STOPPED" ,
8                  "tagCount" : 45 ,
9                  "scanCount" : 128
10             }
11         }
12     }
```

Notes and Caveats

- **No response envelope.** `heartBeatEVT` is a device-emitted event, so it carries **no** `response` / `code` — the command response-code table does not apply. No response code is defined for any event.

`heartBeatEVT` is **read-only telemetry** and carries no credentials. Use it for liveness checks (the *absence* of expected heartbeats is itself a signal), inventory progression, and routine battery monitoring. For the concept-level view, see Monitor Reader Connectivity and Monitor Reader Health and Battery.

Related:  [`config\_events`](#) ·  [`get\_endpoint\_config`](#) ·  What to Monitor and Why · 
[`dataEVT`](#) ·  Monitor Reader Connectivity ·  What to Monitor and Why